

3/4

Review:

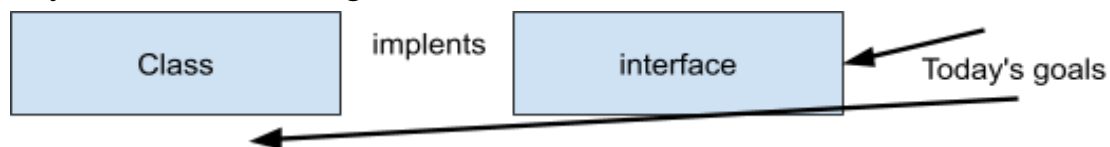
- The “Extends” keyword is used to create an is-a relationship between a subclass and a superclass.
 - A variable with a superclass type can reference a subclass object.
(A variable with subclass type can’t reference a superclass object).
 - Methods of the superclass can be overridden in the subclass(title of the overridden method must be an exact match; re-code it as you like).
 - Which method runs is determined by the type of object at run time - This is called Dynamic Dispatch.
-

Interface:

- Contracts for how an object must behave

Example: If you want to sort objects, they must have a compareToMethod that means their class must satisfy a contract to provide that method.

Key words is used to agree to a contract



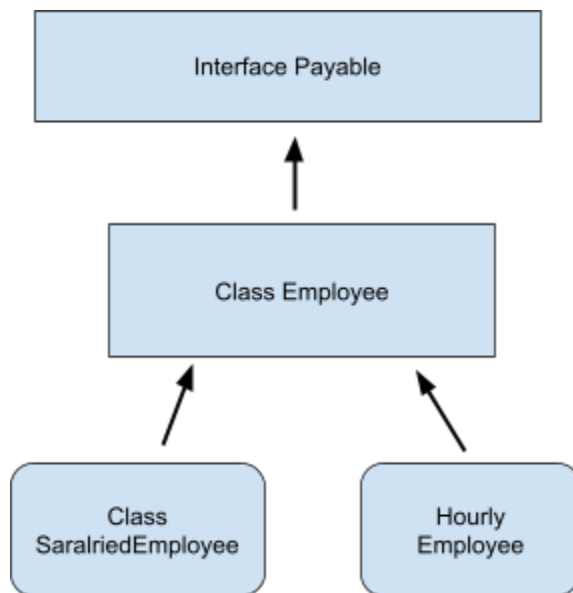
- An interface is a list of required behaviors. Each behavior is a method(title), and it’s without coding. It’s a class with coding omitted.

Suppose we want a guarantee that classes provide a weeklyPay() method.

Public interface Payable {
-- Any class that implements this must provide the method. You can describe variables with an interface for a type, but when you assign objects, those come from implementing. The only things you could do with the variable are the interfaces listed behaviors.

```
    Double weeklyPay;  
→ no constructor - no code  
}
```

- You can declare variables belonging to the interface.



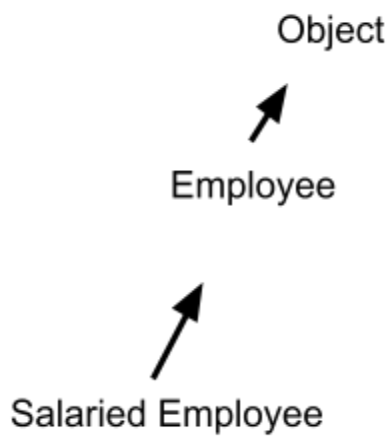
- “Ectends” – Tells us what a subclass is
- Implants - Guarantees what a class will do

Comparable is a standard Java defined interface, it is generic.
ArrayList is generic allow for multiple versions (e,g ArrayList <String>, ArrayList<Employee>, etc
Comparable <String> promises a method int compareTo(String X)
Comparable<Employee> ... int compareTo(Employee X)

A class can implement as many interfaces as you like, but it can only extend one direct superclass.

Imagine:

```
interface Comparable<Employee>{  
    Int compareTo(Employee x);  
}
```



- If a class is comparable, it will be accepted by the Array.sort method.